ARCA

AGENTIC RESEARCH CONTEXT ARCHITECTURE

ARCA V2 · RESEARCH WORKFLOW

Quick Start & User Guide

Your first ARCA project, using prompts instead of shell commands.

01	02	03	04
Connect & set up	Initialize the project	Work with evidence	Checkpoint & resume

> **The idea:** Talk to your coding agent. Let it handle ARCA's files and helper scripts. You approve scientific decisions, costly GPU runs, and commits.

1 · First-time setup (once per server)

1. Connect. Open VS Code → **Remote – SSH** → connect to your compute server. Open the project folder **on the server**. Install/sign in to the **Claude Code** and **Codex** VS Code extensions. Ensure the agent is working in that remote folder.

2. Put the ARCA template in your project. For a new repository, extract the template into a new project directory. For an existing repository, merge ARCA folders carefully instead of overwriting your code or existing instructions. The template already contains your `arca-resume`, `arca-checkpoint`, and `research-evidence-audit` skills; there is no need to reinstall them.

3. Ask one agent to check everything. Paste this prompt into Claude Code or Codex:

> **SETUP CHECK** — I am in a new ARCA project on a remote SSH server. Inspect the repository, the current Git state, the ARCA template, and the available ARCA skills. Check that Claude Code and Codex can discover their project instructions. Report missing or conflicting files. Do not modify anything or run experiments yet.

4. Optional but recommended integrations. If not yet installed in your agent environment, use these *prompts*, one at a time:

CortexDB

> Help me install and configure the official CortexDB plugin (`liliang-cn/cortexdb`) for this coding agent. Use the current official instructions, review requested permissions and hooks, and ask before making changes. Prefer project-isolated memory rather than the default shared brain. Verify recall works. Never copy secrets into memory. Keep Git/ARCA files authoritative.

Feynman

> Help me install the official Feynman research-agent skills in this environment, preferably scoped to this repository. Check the official installation instructions and compatibility with this agent. Preserve existing ARCA skills and instruction files, ask before changes, and verify the research skills are discoverable.

Some extension/plugin installation steps need UI approval or agent tool permissions. A prompt starts the process; it cannot bypass permissions. CortexDB is optional for using ARCA.

2 · Initialize your first project (once per repository)

Paste this into **Claude Code or Codex**:

> **INITIALIZE ARCA** — Read `AGENTS.md`, the applicable agent instructions, and the ARCA template. Help me initialize this new research project. Ask for the research objective, questions, available datasets, evaluation metrics, compute constraints, and target outputs. After I confirm the plan, populate `research/CHARTER.md`, `research/QUESTIONS.md`, `context/STATE.md`, `context/NEXT.md`, and relevant dataset/experiment registries. Do not invent findings. Do not start GPU jobs. Run the repository's ARCA structural check and show me the changes before proposing the first Git commit.

Done when: your research objective and next action are documented; the ARCA check passes or flags understood issues; the initial changes are reviewed and committed.

3 · Daily workflow

When	What to say / invoke	Expected result
Resume	Claude: `/arca-resume` · Codex: `$arca-resume`	Read-only reconstruction of current state and next steps
Start experiment	"Create an ARCA experiment for **[hypothesis]**. Define controls, baseline, metrics, seeds, and a pilot. Register it, but wait for my approval before GPU use."	Agent invokes `new_experiment.py`, completes the protocol; no training yet
Monitor a run	"Check the current job, GPU use, ETA, first results, and anomalies. Report any issue early."	Progress report; no unattended scientific conclusions
Audit evidence	Claude: `/research-evidence-audit` · Codex: `$research-evidence-audit`	Trace claim → metrics/config/data/artifacts; flag gaps
Validate ARCA	"Run the ARCA structural check; summarize failures. Don't fix automatically."	Agent invokes `arca_check.py`
Checkpoint	Claude: `/arca-checkpoint` · Codex: `$arca-checkpoint`	Agent updates STATE/NEXT and relevant notes, then invokes `session_checkpoint.py`

Typical day: Resume → Plan → Approve → Run & monitor → Review evidence → Checkpoint.

4 · What is automatic (and what isn't)?

Feature	Reality in ARCA v2
`arca-resume`	The skill guides the agent to read context; invoke it explicitly for reliable behavior.
`new_experiment.py`	Creates ID and starter files; the agent must fill in and later update the experiment.
`arca_check.py`	Checks structure and selected references, **not** scientific correctness or CortexDB connectivity.
`arca-checkpoint`	The skill guides updating notes **before** invoking the snapshot script.
Hooks	Only run if installed/enabled for that agent; the template includes examples, not guaranteed automatic lifecycle management.

Never automatic without your approval: expensive shared-GPU jobs, major methodological changes, publication conclusions, or Git commits.

5 · FAQ

Do I need to manually run Python scripts?
No. Ask Claude Code or Codex to perform the operation; the agent can call the appropriate script. Review its planned actions.

Do I have to fill in every Markdown file myself?
No. Start with the *INITIALIZE ARCA* prompt. The agent should populate only relevant files, asking you for missing scientific facts rather than inventing them.

How do I resume after a new session, compaction, or model switch?
Invoke `arca-resume` in the new session. It reads the canonical repository state; it does not need the previous chat. Resume quality depends on previous checkpoint quality.

Does closing VS Code stop GPU training?
Not if the job was launched as a server-side persistent process or scheduled job. Check your actual process/scheduler setup; ARCA alone does not keep jobs alive.

What if Claude and Codex both work on the same project?
Use separate tasks or worktrees/file ownership. For independent validation, have one agent review the other's result without editing the same files at the same time.

Is CortexDB required? What if it remembers outdated information?
No. CortexDB is optional retrieval memory. Check anything consequential against Git-tracked ARCA records and experiment artifacts.

What if the ARCA check passes but an experiment is wrong?
That's possible: a structural check is not an experiment audit. Use `research-evidence-audit` and appropriate tests.

When should I checkpoint?
Before ending a substantial work session, switching agents/models, or risking compaction. Checkpoint before a long interruption even if the experiment is incomplete.

I already have a repository: do I start over?
No. Ask the agent to merge the ARCA scaffolding without overwriting existing code, configs, data, or instructions; review changes first.

Remember: *ARCA records are the source of truth. CortexDB helps you find them. The agents do the work; you make the research decisions.*